

Faculty of Computers and Information Systems

Information Retrieval
Practical Laboratory

Week 8: Vector Space Model and Cosine Similarity

Represent a query in the same TF-IDF space as the documents, then rank the collection by cosine similarity. Compute every score from the text.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Construct a query vector using the document vocabulary and IDF.
- Implement dot product, magnitude, and cosine similarity manually.
- Rank documents by decreasing similarity with deterministic ties.
- Handle zero vectors and out-of-vocabulary query terms.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

In the vector space model, a document and a query are vectors over the same vocabulary. The query uses its own raw term counts multiplied by the IDF values learned from the document collection. Do not add the query to the collection or recalculate IDF for each search.

The dot product sums products of matching coordinates. The magnitude is the square root of the sum of squared coordinates. Cosine similarity divides the dot product by both magnitudes. It measures direction agreement rather than raw vector length. With these nonnegative weights, scores range from zero to one, allowing for tiny floating-point error.

The formula is undefined if either vector has magnitude zero. The implementation returns 0.0 by explicit convention. Empty, stop-word-only, or entirely unknown queries therefore produce no positive score. Terms absent from the vocabulary are ignored when constructing a query vector.

The teaching program prints all documents, including zero scores. Week 9 evaluates positive-score results at a fixed cutoff. Equal scores are ordered by document ID so reruns agree. A rank is a computed order, not a guarantee that a document is relevant to a human information need.

$$q \cdot d = \sum_i q_i d_i$$

$$||v|| = \sqrt{(\sum_i v_i^2)}$$

$$\cos(q, d) = \frac{q \cdot d}{||q|| ||d||}$$

Course material: Extension of Week 7 TF-IDF; the ten-document collection and preprocessing are unchanged.

3. Python Concepts Used

- **sum and indexing:** Accumulate products of aligned coordinates.
- **math.sqrt:** Compute vector magnitude.
- **Functions:** Keep dot product, magnitude, and cosine calculations separate.
- **Tuples and sorting:** Store (document ID, score) and sort by negative score, then ID.
- **Command-line arguments:** Accept a custom free-text query without changing source code.

4. Worked Example

Calculate the similarity between $q = [1, 1]$ and $d = [1, 0]$ by hand. Their dot product is 1, their magnitudes are $\sqrt{2}$ and 1, and their cosine is $1/\sqrt{2}$.

```
import math
q = [1.0, 1.0]
d = [1.0, 0.0]
dot = sum(q[i] * d[i] for i in range(len(q)))
q_size = math.sqrt(sum(value * value for value in q))
d_size = math.sqrt(sum(value * value for value in d))
score = dot / (q_size * d_size)
print(f"{score:.4f}")
```

The output is 0.7071. Scaling d to $[2, 0]$ leaves its direction and cosine unchanged. The full program adds a zero-denominator guard and checks equal vector lengths. Its default query is python data analysis, and all scores below are taken from the actual ten-document run.

5. Step-by-Step Practical Task

Step 1: Reuse the Week 7 document collection, vocabulary, and IDF calculations.

Step 2: Implement dot_product, magnitude, and cosine_similarity in separate functions.

Step 3: Tokenize the query and count query term frequencies.

Step 4: Construct the query vector in the existing vocabulary order, using the collection IDF.

Step 5: Calculate a score against every document, then sort by decreasing score and increasing ID for ties.

Step 6: Run the default query and at least two custom queries. Test empty-weight queries and record why they have zero scores.

Execution: open a terminal in this week folder and run `python lab08_solution.py`. On Windows, `py -3` can replace `python`. All required files are included; no internet or third-party Python packages are needed.

For one custom query, run `python lab08_solution.py "python data analysis"`.

6. Complete Python Program

The listing below matches `lab08_solution.py` exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab08_solution.py

```
import os
import re

STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

SAMPLE_DOCUMENTS = {
    "doc1": "Python is great for data science and machine learning",
    "doc2": "Java is used for enterprise applications and web development",
    "doc3": "Machine learning uses neural networks and decision trees",
    "doc4": "Python supports data analysis with libraries like Pandas",
    "doc5": "Java and C++ are used for game development and graphics",
}

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    """Load UTF-8 .txt files; use five samples if missing or empty."""
    documents = {}
```

```

    if os.path.isdir(folder_path):
        for filename in sorted(os.listdir(folder_path)):
            filepath = os.path.join(folder_path, filename)
            if filename.endswith(".txt") and os.path.isfile(filepath):
                with open(filepath, "r", encoding="utf-8") as file:
                    documents[filename[:-4]] = file.read()
    if not documents:
        return SAMPLE_DOCUMENTS.copy()
    return documents

def tokenize(text):
    """Use the same ASCII-letter tokens as the supplied course files."""
    words = re.findall(r"[a-z]+", text.lower())
    return [word for word in words if word not in STOP_WORDS]

import math

def term_frequency(tokens):
    counts = {}
    for term in tokens:
        counts[term] = counts.get(term, 0) + 1
    return counts

def build_tfidf(documents):
    """Raw TF and natural-log IDF; no smoothing or extra normalization."""
    counts = {}
    document_frequency = {}
    for name, text in documents.items():
        counts[name] = term_frequency(tokenize(text))
        for term in counts[name]:
            document_frequency[term] = document_frequency.get(term, 0) + 1
    vocabulary = sorted(document_frequency)
    idf = {}
    total = len(documents)
    for term in vocabulary:
        idf[term] = math.log(total / document_frequency[term])
    vectors = {}
    for name in sorted(documents):
        vectors[name] = [
            counts[name].get(term, 0) * idf[term] for term in vocabulary
        ]
    return vocabulary, counts, document_frequency, idf, vectors

def dot_product(v1, v2):
    if len(v1) != len(v2):
        raise ValueError("Vectors must have the same length")
    return sum(v1[i] * v2[i] for i in range(len(v1)))

def magnitude(vector):
    return math.sqrt(sum(value * value for value in vector))

def cosine_similarity(v1, v2):
    product = dot_product(v1, v2)
    denominator = magnitude(v1) * magnitude(v2)
    if denominator == 0:
        return 0.0
    return product / denominator

def query_vector(query, vocabulary, idf):
    counts = term_frequency(tokenize(query))
    return [counts.get(term, 0) * idf[term] for term in vocabulary]

def rank_query(query, vocabulary, idf, vectors):
    vector = query_vector(query, vocabulary, idf)
    results = []
    for name, document_vector in vectors.items():
        score = cosine_similarity(vector, document_vector)
        results.append((name, score))

```

```

# Stable, reproducible order for equal scores.
results.sort(key=lambda item: (-item[1], item[0]))
return results

def main():
    import sys
    documents = load_documents(os.path.join(BASE_DIR, "docs"))
    vocabulary, counts, df, idf, vectors = build_tfidf(documents)
    query = " ".join(sys.argv[1:]) or "python data analysis"
    print("Query:", query)
    vector = query_vector(query, vocabulary, idf)
    print("Query magnitude:", f"{magnitude(vector):.4f}")
    if magnitude(vector) == 0:
        print("No indexed query terms with nonzero weight.")
    print("Ranking:")
    for position, (name, score) in enumerate(
        rank_query(query, vocabulary, idf, vectors), 1
    ):
        print(f"{position:2}. {name:6} {score:.4f}")

if __name__ == "__main__":
    main()

```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

```

Query: python data analysis
Query magnitude: 2.3793
Ranking:
1. doc4    0.5123
2. doc9    0.4326
3. doc1    0.3466
4. doc6    0.0489
5. doc7    0.0284
6. doc10   0.0000
7. doc2    0.0000
8. doc3    0.0000
9. doc5    0.0000
10. doc8   0.0000

```

These results use all ten shipped documents. Editing the collection changes the index and may change weights, ranks, and evaluation results.

8. Laboratory Exercises

1. Hand calculation: compute cosine for [1, 2] and [2, 1], then verify it with your function.
2. Implementation: complete the starter and add explicit handling of zero vectors.
3. Debugging: reorder one vector without reordering the other. Explain why the resulting dot product is meaningless.
4. Interpretation: compare queries python data and python python data. Explain how query TF changes the direction.
5. Implementation: verify that $\text{cosine}(v, v)$ is approximately 1 for a nonzero vector.
6. Extension: print the nonzero query coordinates and explain the strongest term contributions for the top document.

9. Student Tasks

- Completed vector functions with normal, zero-vector, and length-mismatch tests.

- Computed rankings for three queries using the unchanged collection.
- A short explanation of query weighting, tie order, and out-of-vocabulary handling.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. Why must query and document coordinates refer to the same terms?
2. Why should query processing reuse collection IDF?
3. What happens when either vector has magnitude zero?
4. How does multiplying a nonzero vector by a positive scalar affect cosine?
5. Why is a high similarity score not itself a relevance judgment?

11. Challenge Exercise

Optional: Implement sparse vectors as dictionaries from terms to weights and compare scores with the dense baseline. Measure storage and calculation counts on the same collection.